

VOICE AI SUPPORT ENGINEER

Complete Codex implementation prompt

A zero-cost-first portfolio demo with RAG, explicit tools, observable workflows, Gemini Live voice, and a clean OpenAI Realtime adapter boundary.

DEFAULT VOICE	Gemini Live / Native Audio
SECOND PROVIDER	OpenAI Realtime
FIRST DELIVERY	Runnable text-based vertical slice
CORE STACK	TypeScript, Next.js, PostgreSQL, pgvector

Copy the following prompt into Codex at the root of the target repository.

I want to build a polished pet project / portfolio demo: a Voice AI Support Engineer for a fictional B2B telecom/CPaaS company.

The goal is not merely to add voice to a chatbot. I want this demo to showcase the capabilities clients typically expect from production Voice AI systems:

- real-time voice conversation
- low latency and interruption / barge-in
- RAG over company documentation
- tool/function calling
- customer and account context
- multi-step troubleshooting
- structured call outcomes
- support ticket creation
- human escalation
- guardrails and confirmation before sensitive actions
- conversation and customer memory
- observability of everything the agent does
- a polished live dashboard suitable for showing to potential clients
- swappable realtime voice/LLM providers

I want to keep the demo as close to zero operating cost as practical. Use Gemini Live / Gemini Native Audio as the initial realtime voice provider. However, the realtime layer **MUST** be behind a clean provider abstraction so OpenAI Realtime can be added and selected later without changing RAG, tools, domain logic, the dashboard, support workflows, or the core agent runtime.

Do not turn this abstraction into a universal AI-provider framework. It only needs to handle the real differences between Gemini Live and OpenAI Realtime cleanly.

Product concept

The demo represents a fictional B2B telecom/CPaaS provider. A customer opens the web application, starts a browser voice call, and speaks to an AI technical support engineer.

Example customer request:

Hi, our outbound calls started failing this morning. We're getting SIP 403 responses when calling UK numbers.

The agent should behave like a support engineer connected to real internal systems, not a generic chatbot. It should be able to:

1. Identify the customer from the caller/session.
2. Retrieve account information.
3. Search the knowledge base using RAG.
4. Inspect recent calls.
5. Inspect individual call details and SIP responses.
6. Check trunk configuration.
7. Check number configuration.
8. Check current service incidents.
9. Reason from retrieved information.
10. Explain the likely cause to the customer.

11. Create a support ticket when necessary.
12. Escalate to a human engineer when appropriate.
13. Send a follow-up or schedule a callback.
14. Produce a structured call outcome.

Tool system

Initial tools may include:

- get_customer
- get_account
- get_recent_calls
- get_call_details
- check_trunk_status
- check_number_configuration
- get_service_incidents
- search_knowledge_base
- create_support_ticket
- send_followup
- schedule_callback
- escalate_to_engineer

For the first version, operational systems should be realistic mocked services backed by PostgreSQL. I care much more about demonstrating sound architecture and realistic agent behavior than integrating actual CRM, ticketing, and carrier systems immediately.

Tools must be defined independently of Gemini- or OpenAI-specific function-calling formats. Conceptually:

```
interface ToolDefinition<TInput, TOutput> {  
  name: string;  
  description: string;  
  inputSchema: Schema<TInput>;  
  permission: "read-only" | "write" | "sensitive-write" | "human-only";  
  execute(input: TInput, context: ToolContext): Promise<TOutput>;  
}
```

Provider adapters may translate these definitions into provider-native schemas, but provider SDK types must not leak into tool implementations. Tool execution must be observable and independently testable.

Realtime voice architecture

Use Gemini Live / Gemini Native Audio as the initial/default realtime provider. The conceptual architecture is:

```

Browser
|
| realtime audio
v
Voice Session
|
v
RealtimeVoiceProvider
|
+-- GeminiLiveProvider      (initial/default)
+-- OpenAIRealtimeProvider (later)
|
v
Agent Runtime
+-- Tool Registry / Executor
+-- RAG
+-- Customer Context
+-- Memory
+-- Guardrails
+-- Event Bus / Observability

```

Do not couple application logic to Gemini SDK types. Define a small internal realtime event model. Something approximately like:

```

interface RealtimeVoiceProvider {
  connect(config: SessionConfig): Promise<RealtimeVoiceSession>;
}

interface RealtimeVoiceSession {
  sendAudio(chunk: ArrayBuffer): void;
  sendToolResult(result: ToolExecutionResult): Promise<void>;
  interrupt(): Promise<void>;
  close(): Promise<void>;

  on(event: "audio", handler: AudioHandler): void;
  on(event: "transcript", handler: TranscriptHandler): void;
  on(event: "toolCall", handler: ToolCallHandler): void;
  on(event: "turnStarted", handler: TurnHandler): void;
  on(event: "turnEnded", handler: TurnHandler): void;
  on(event: "error", handler: ErrorHandler): void;
}

```

This interface is illustrative. Improve it after studying the actual provider APIs. Do not blindly force both providers into an abstraction that discards useful capabilities. Keep a small common interface and permit provider-specific configuration/extensions where genuinely necessary.

Provider selection should be configuration-driven:

```
VOICE_PROVIDER=gemini
```

and later:

```
VOICE_PROVIDER=openai
```

Switching providers must not require changes to RAG, tools, the database, customer models, memory, the dashboard event model, or support workflows.

Voice behavior

The realtime implementation should eventually support:

- browser microphone input
- streaming audio input and output
- interruption / barge-in
- turn detection

- tool calls during conversation
- tool results returned into the live conversation
- graceful latency handling
- reconnect and error handling
- conversation state management
- transcripts when available

Prefer browser/WebRTC where the provider supports it appropriately. If Gemini Live and OpenAI Realtime require different browser/backend transport topologies, isolate the difference within the voice/provider layer instead of pretending their transports are identical. PSTN through Twilio or Telnyx can come later.

Agent architecture

Do NOT create an unnecessary multi-agent framework. Start with one primary support agent.

The application, rather than the model provider, should explicitly own:

- customer context
- tool registry and execution
- RAG
- permissions and guardrails
- support workflow state
- structured outcomes
- application-level memory
- observability

The realtime model handles the conversation and decides when tools are appropriate. It must not become the database or business-logic layer. Business workflows, permissions, and state transitions must remain explicit and testable.

Guardrails and confirmation

Sensitive or mutating operations must support confirmation requirements. Example:

Agent: I can reset your trunk credentials. This will invalidate the current credentials. Would you like me to proceed?

Only after explicit confirmation should the operation execute. Support a simple permission classification:

- read-only
- write
- sensitive-write
- human-only

Do not build a large authorization framework yet. Create a clean extension point and test at least one confirmation-gated operation.

RAG

Implement a proper retrieval pipeline rather than putting all documents into the system prompt:

```
documents
-> parsing
-> chunking
-> embeddings
-> PostgreSQL + pgvector
-> hybrid retrieval
-> optional reranking
-> selected chunks + metadata
-> agent context
```

Use a local/free embedding model where practical, such as a small modern BGE/E5-style model suitable for an English technical-support knowledge base. Choose something reasonably small, easy to run locally, and operationally simple. Do not introduce Ollama unless it provides a concrete benefit; direct inference from the application or a small local service is acceptable.

Prefer pragmatic retrieval:

- pgvector semantic search
- PostgreSQL full-text or BM25-like lexical retrieval where practical
- combined ranking
- optional lightweight reranking

Do not over-engineer retrieval before measurements show it is needed. Every retrieved chunk should expose useful metadata:

- document
- section
- chunk ID
- semantic/lexical/combined scores
- source/type

The dashboard must show retrieved sources and scores.

Knowledge base

Create realistic fictional telecom documentation covering:

- SIP response-code troubleshooting
- outbound calling troubleshooting
- trunk configuration and authentication
- caller-ID requirements
- international calling restrictions
- UK calling guide
- number configuration
- service plans
- incident handling
- escalation procedures

Eventually support dashboard document upload:

```
Upload PDF / Markdown / text
-> parse
-> chunk
-> embed
-> index
-> immediately searchable by the agent
```

This is an important demo feature because it makes RAG behavior visually obvious to non-technical clients.

Memory

Design application-level memory separately from raw conversation history. Eventually support:

- customer facts
- customer preferences
- previous support cases
- conversation summaries
- important extracted facts
- relevant historical context

Do not keep sending every historical transcript to the model. Memory retrieval must be selective:

```
new call
-> identify customer
-> retrieve relevant previous cases
-> retrieve customer facts
-> retrieve recent conversation summaries
-> provide a small relevant context
```

The realtime provider's session context is not the long-term memory system.

Dashboard

Build a polished Next.js/React dashboard suitable for recording demo videos and showing directly to clients.

Left: call

- customer/caller
- call status and duration
- microphone and call controls
- live transcript
- user and agent turns

Center: agent activity

Show an observable activity timeline, for example:

```
Customer identified
Acme Ltd

Searching knowledge base
"SIP 403 outbound UK"

Retrieved 4 chunks
- UK Calling Troubleshooting - 0.91
- SIP Response Codes - 0.87

Checking recent calls...
3 failed calls found.

Checking trunk configuration...
Configuration valid.

Checking service incidents...
UK carrier degradation detected.

Creating support ticket...
SUP-1042 created.
```

IMPORTANT: Do not expose, request, or reconstruct private model chain-of-thought. The timeline contains only observable application events: retrieval queries/results, tool calls, sanitized inputs, results, state transitions, confirmations, timings, errors, provider events, and structured outcomes.

Right: context

- customer/account and plan
- active products
- retrieved knowledge
- current tools/actions
- current diagnosis
- support case
- escalation state
- call outcome

After-call view

After a call, show:

- full transcript
- summary
- detected intent and severity
- affected product
- diagnosis
- resolution status
- actions performed
- ticket ID and next action
- retrieved sources
- tool calls
- RAG and tool timings
- voice/model latency metrics where available

The UI should reveal how the system works instead of hiding everything behind a chat box.

Event and observability architecture

Create a provider-independent application event model, approximately:

```
type AgentEvent =  
  | CustomerIdentifiedEvent  
  | RetrievalStartedEvent  
  | RetrievalCompletedEvent  
  | ToolCallStartedEvent  
  | ToolCallCompletedEvent  
  | ConfirmationRequiredEvent  
  | SupportStateChangedEvent  
  | TranscriptEvent  
  | VoiceTurnEvent  
  | CallOutcomeEvent  
  | ErrorEvent;
```

Events drive the live dashboard and are persisted for post-call inspection. Include timestamps, correlation IDs, and durations where relevant. Eventually a completed call should expose a timing sequence such as:

```
user finished speaking  
+120 ms final transcript/turn  
+210 ms model requested tool  
+35 ms tool execution  
+90 ms model resumed  
+180 ms first audio response
```

Exact metrics depend on provider capabilities, but design for this level of observability.

Structured call outcome

Use a proper runtime-validated schema rather than arbitrary JSON. Example:

```
{  
  "customer": "Acme Ltd",  
  "intent": "technical_support",  
  "severity": "high",  
  "product": "SIP Trunking",  
  "issue": "Outbound UK calls return SIP 403",  
  "diagnosis": "Upstream carrier incident",  
  "resolved": false,  
  "actions": [  
    "searched_knowledge_base",  
    "checked_recent_calls",  
    "checked_service_incidents",  
    "created_support_ticket"  
  ],  
  "ticketId": "SUP-1042",  
  "nextAction": "Monitor carrier incident"  
}
```

Demo data

Create a fictional telecom/CPaaS company with deterministic seed data.

Main scenario:

```
Customer: Acme Ltd
Plan: Business
Products: SIP Trunking
International calling: enabled
UK calling: enabled

Recent calls:
- successful historical calls
- several recent UK calls returning SIP 403

Trunk:
- registered/configured correctly
- credentials valid

Current incident:
- UK outbound carrier degradation
- status: investigating
```

The knowledge base must contain enough information for the model to distinguish common SIP 403 causes: invalid credentials, caller-ID restrictions, international calling disabled, destination restrictions, carrier rejection, and an active carrier incident.

Expected successful demo flow:

```
Customer reports SIP 403
-> retrieve customer
-> inspect recent calls
-> confirm repeated UK failures
-> search relevant documentation
-> inspect account/trunk configuration
-> confirm configuration appears correct
-> inspect service incidents
-> find UK carrier incident
-> explain diagnosis
-> create support ticket
-> produce structured outcome
```

The scenario must be deterministic enough to demonstrate repeatedly. Later add scenarios for:

1. Caller-ID misconfiguration.
2. International calling disabled.
3. Invalid trunk credentials.
4. Account/balance restriction.
5. Number configuration issue.
6. No known diagnosis, requiring human escalation.

These scenarios must reuse the same tools rather than hard-coded conversation scripts.

Cost constraints

This is a pet/demo project and runtime cost should stay near zero:

- Gemini Live is the default realtime provider.
- Prefer free/local embeddings.
- Run PostgreSQL + pgvector locally through Docker.
- Avoid paid managed infrastructure.
- Avoid dependencies requiring paid SaaS accounts for basic development.
- Mock external CRM, ticketing, and telecom systems locally.
- Do not introduce Redis unless it solves a demonstrated need.
- Do not introduce Kubernetes.
- Do not split the project into microservices without a concrete reason.

Local startup should be as simple as practical:

```
docker compose up -d
pnpm install
pnpm db:migrate
pnpm db:seed
pnpm dev
```

plus Gemini credentials/environment configuration.

Preferred stack

- TypeScript / Node.js
- pnpm
- Next.js / React
- PostgreSQL + pgvector
- Docker Compose
- Gemini Live / Native Audio
- WebRTC and/or WebSocket where appropriate
- Zod or equivalent runtime schemas
- structured logging

Use another technology only when there is a concrete advantage.

Repository structure

Choose a sensible monorepo structure, approximately:

```
apps/
  web/
  api/
packages/
  agent/
  voice/
  rag/
  tools/
  db/
  shared/
infra/
docs/
```

Change this if a better structure follows from the existing repository. Keep boundaries meaningful; do not create packages for tiny amounts of code.

Engineering requirements

This should be production-style code even though it is a demo. Priorities:

- strong TypeScript types
- clean module boundaries
- dependency injection where useful
- structured logging and observability
- error handling
- database migrations
- deterministic seed data

- testable tools
- unit and integration tests for important workflows
- environment-based configuration
- Docker-based local infrastructure
- straightforward README/setup
- realistic domain modeling

Avoid:

- giant service classes
- excessive abstraction
- unnecessary microservices
- premature Kubernetes
- generic AI framework abstractions
- fake complexity added only to look enterprise
- business logic hidden entirely inside prompts
- provider-specific SDK types leaking throughout the application

Development strategy

Do not attempt to implement the entire system in one giant pass. Work in runnable, testable vertical slices.

Milestone 1 - Agent + tools + RAG

Text interaction is acceptable initially. Build an end-to-end slice:

```
browser text conversation
-> agent
  +-> seeded customer/account
  +-> RAG
  +-> telecom support tools
  +-> service incident lookup
  +-> ticket creation
-> structured outcome
-> live dashboard activity timeline
```

This validates the domain model, tool architecture, RAG, agent runtime, events/observability, UI, and structured outcomes before realtime audio debugging. The milestone **MUST** produce something I can run and interact with.

Milestone 2 - Gemini Live voice

Add:

- Gemini Live / Native Audio provider
- browser microphone input
- streaming audio playback
- interruption/barge-in
- realtime tool calling and tool-result return
- transcript integration
- voice/provider events
- latency metrics

The existing tools, RAG, domain model, and workflow code should not require significant changes.

Milestone 3 - Knowledge ingestion + memory

Add:

- document upload
- parsing and chunking
- local embeddings and indexing
- retrieval visualization
- customer facts
- conversation summaries
- previous-case retrieval

Milestone 4 - OpenAI Realtime provider

Implement OpenAIRealtimeProvider against the same internal realtime interface. Allow:

```
VOICE_PROVIDER=gemini
```

or:

```
VOICE_PROVIDER=openai
```

Run the same scripted support scenario through both providers. Document capability differences, transport differences, configuration, latency observations, and any intentionally provider-specific behavior. Provider selection must not fork the application architecture.

Milestone 5 - Production-style demo polish

Add the highest-value portfolio features:

- polished responsive dashboard
- post-call inspection view
- reusable demo scenario selector/reset
- confirmation-gated sensitive action
- failure and recovery states
- human-escalation flow
- optional PSTN adapter via Twilio/Telnyx
- deployment documentation
- a short scripted demo path suitable for a recorded video

Acceptance criteria for the first vertical slice

Milestone 1 is complete only when all of the following are true:

1. A developer can start the stack from the README.
2. Deterministic seed data is installed by a documented command.
3. The browser offers a text conversation with the support agent.
4. The Acme SIP-403 scenario causes real tool executions through the tool registry.
5. RAG retrieves seeded telecom documentation from PostgreSQL/pgvector.
6. Incident lookup returns the seeded UK degradation.
7. A support ticket is persisted, not merely mentioned by the model.

8. A validated structured outcome is persisted and displayed.
9. Observable events appear live in the dashboard timeline.
10. Important tools and the main scenario have automated tests.
11. No Gemini/OpenAI provider SDK types leak into tools, RAG, or domain modules.
12. The README clearly states what is real, what is local, and what remains mocked.

How I want us to work

First inspect the repository and environment if one already exists. Before substantial implementation:

1. Propose the concrete architecture and important tradeoffs.
2. Write a concise architecture document into the repository.
3. Define the domain model and main interfaces.
4. Define the first vertical slice and acceptance tests.
5. Identify any assumptions or decisions that genuinely require my input.

Then proceed unless blocked by one of those decisions:

6. Create the project skeleton.
7. Implement Milestone 1 end to end.
8. Run migrations and seed data.
9. Run the application and tests.
10. Verify the UI and main deterministic scenario.
11. Report exactly what works, what remains mocked, commands to run it, and the next recommended increment.

Do not stop after writing only a plan if implementation can safely proceed. Do not implement every future milestone at once. The first deliverable must be a coherent, runnable vertical slice rather than a collection of disconnected scaffolds.